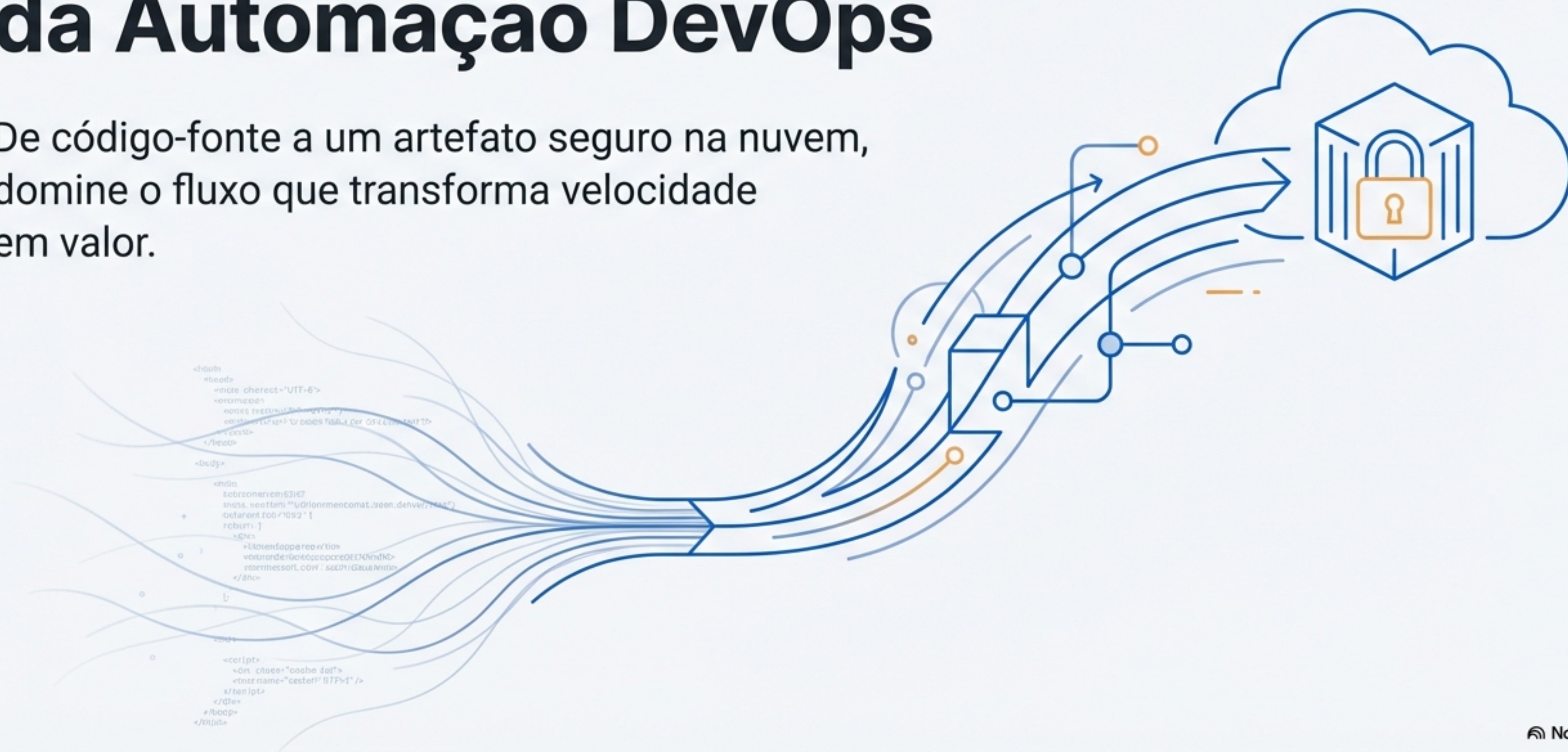


CI/CD e ECR: A Jornada da Automação DevOps

De código-fonte a um artefato seguro na nuvem,
domine o fluxo que transforma velocidade
em valor.



Os Princípios Fundamentais: CI e CD, a Dupla da Qualidade e Velocidade



CI - Integração Contínua

A prática de automatizar a compilação e os testes do código sempre que mudanças são enviadas ao repositório.

Contexto de Containers

- Automatiza o processo de `build` da imagem Docker.
- Executa testes unitários e de integração.
- A imagem Docker se torna o artefato final validado.

Objetivo: Detectar problemas rapidamente, garantindo que o código está sempre em estado funcional.



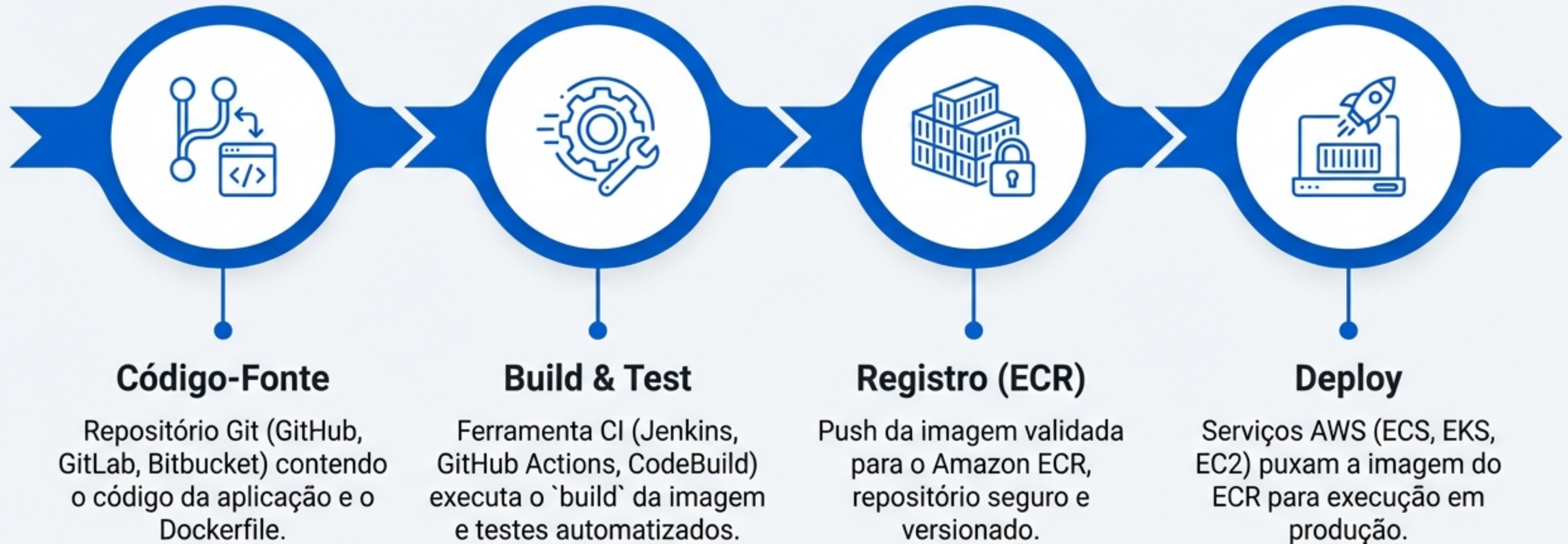
CD - Entrega Contínua

Complementa o CI, automatizando a distribuição do artefato validado para ambientes de destino.

Benefícios

- Automatiza a entrega da imagem para repositórios.
- Facilita deploys em ambientes de *staging* e produção.
- Reduz o tempo entre desenvolvimento e entrega.

O Mapa da Jornada: A Anatomia do Pipeline DevOps



Este fluxo representa a jornada completa de uma aplicação, garantindo qualidade e rastreabilidade em cada etapa.

O Destino: Amazon ECR, a Fortaleza Segura para suas Imagens

O Amazon ECR é o repositório Docker totalmente gerenciado pela AWS, projetado para armazenar, gerenciar e implantar imagens de container de forma segura e escalável.



Totalmente Gerenciado

Não é necessário provisionar ou gerenciar servidores de registro. Alta disponibilidade com replicação automática.



Integração Nativa

Funciona perfeitamente com ECS, EKS, Lambda e outros serviços AWS. Políticas granulares de IAM definem quem pode fazer `push` ou `pull`.



Segurança Robusta

Criptografia em repouso e em trânsito. Análise automática de imagens em busca de vulnerabilidades conhecidas (Scanning).

Passo 1: O Pedido de Acesso (Autenticação)

Antes de enviar uma imagem para o ECR, é necessário autenticar o cliente Docker local com o registro da AWS, usando credenciais temporárias geradas pelo AWS CLI.



A Chave Mestra

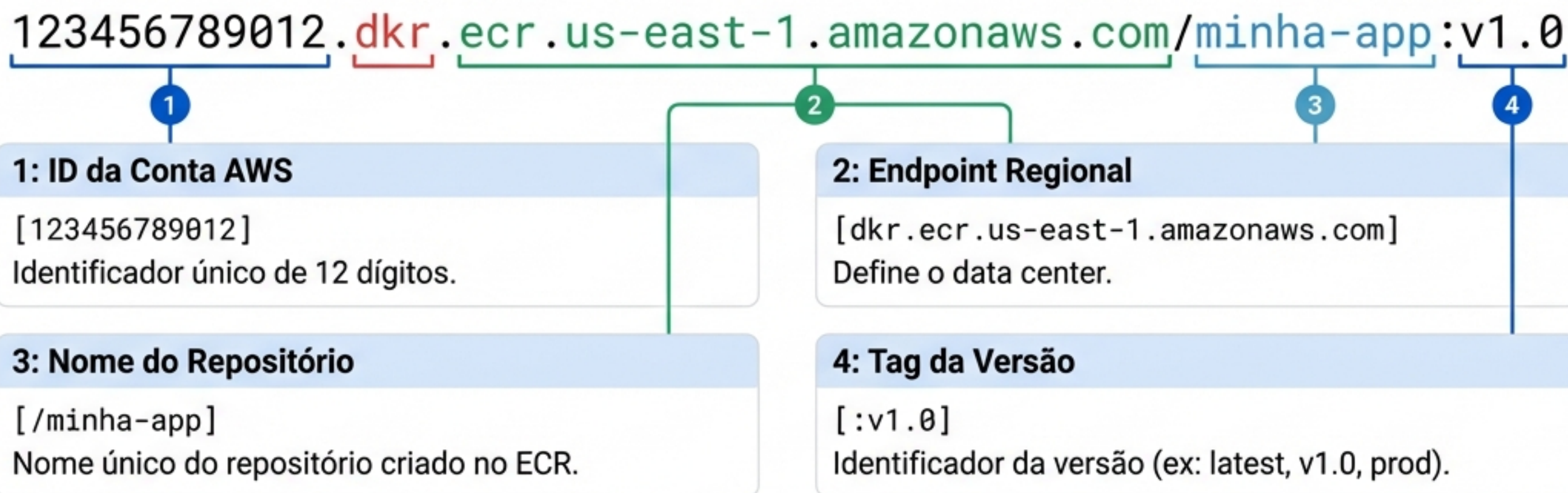
```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin  
123456789012.dkr.ecr.us-east-1.amazonaws.com
```

Este comando combina a geração de senha com o login em uma única operação elegante, sem expor a senha no histórico de comandos.

Passo 2: A Identidade da Imagem (Tagging)

O processo de tagging é crucial para preparar a imagem local para o upload ao ECR. A imagem precisa ser renomeada seguindo a convenção de URI específica da AWS.

Anatomia do URI do ECR



Exemplo Prático de Tagging

```
docker tag minha-app:latest 123456789012.dkr.ecr.us-east-1.amazonaws.com/minha-app:v1.0
```

Este comando cria um alias para a imagem local `minha-app:latest`, apontando-a para o URI completo do ECR. A imagem original permanece inalterada.

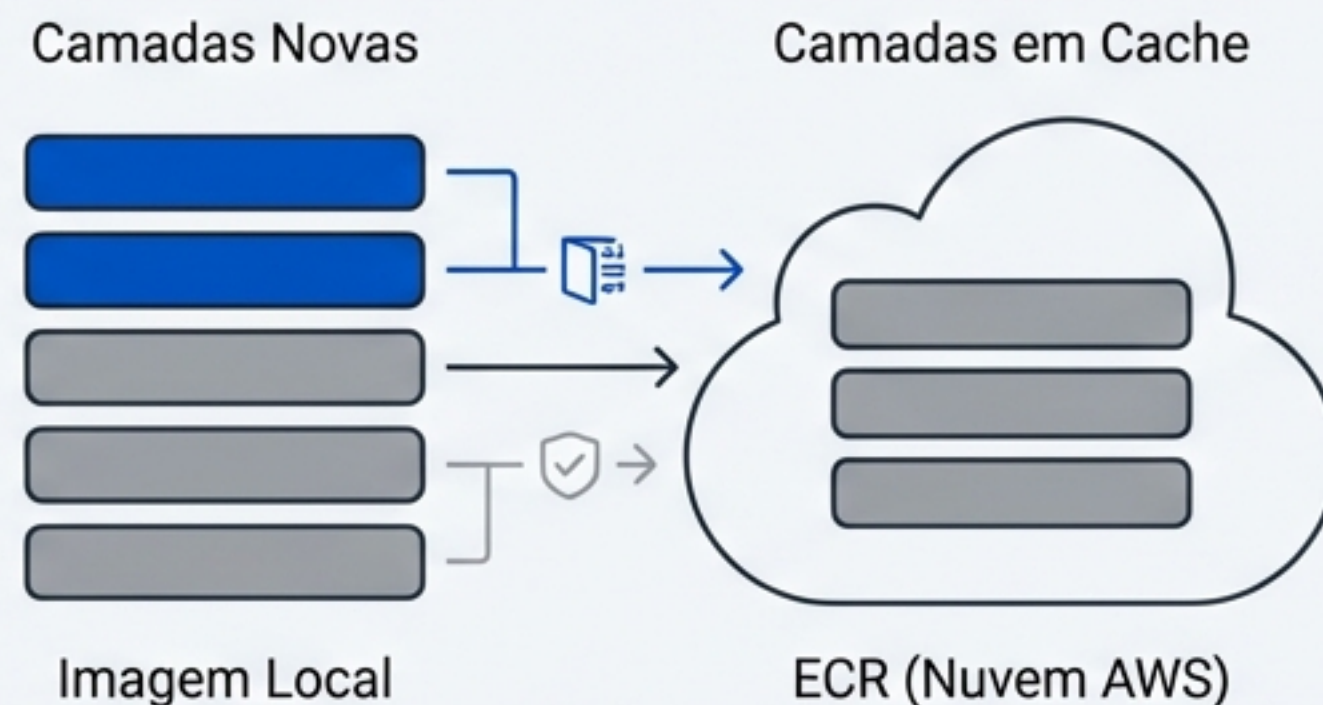
Passo 3: A Ascensão para a Nuvem (Push)

Com a imagem devidamente 'taggeada' e a autenticação estabelecida, o último passo é enviar a imagem para o repositório ECR na nuvem AWS.

```
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/minha-app:v1.0
```

O Que Acontece nos Bastidores

- **Cache Inteligente:** Docker identifica quais camadas já existem no ECR (cache).
- **Transferência Otimizada:** Apenas camadas novas ou modificadas são transferidas.
- **Compressão e Paralelismo:** Cada camada é comprimida e enviada em paralelo.
- **Integridade Garantida:** Checksums garantem a integridade durante a transferência.



Missão Cumprida: Sua Imagem Reside na Nuvem

Sua imagem Docker agora existe como um artefato de primeira classe no ecossistema AWS.



Versionada

Identificada unicamente por tag e digest SHA256.



Disponível

Acessível por qualquer serviço AWS autorizado na região.



Segura

Criptografada e protegida por políticas IAM.



Rastreável

Logs completos de push, pull e scanning de vulnerabilidades.

O Próximo Capítulo: Orquestração e Deploy

Com a imagem seguramente armazenada no ECR, o próximo passo lógico é entender como os serviços de orquestração da AWS consomem esse artefato para executar containers em produção.



ECR: Fonte da Verdade

O repositório ECR atua como registro central onde todas as versões validadas residem.



Serviços de Orquestração

ECS, EKS ou EC2 recebem instruções para executar containers baseados em imagens do ECR.



Pull Automatizado

O serviço autentica via IAM roles e faz pull da imagem especificada.



Execução do Container

A imagem é instanciada como container rodando em infraestrutura gerenciada.

Amazon ECS

Orquestração proprietária da AWS, totalmente gerenciado.

Amazon EKS

Kubernetes gerenciado pela AWS, compatível com ecossistema K8s.

EC2 Puro

Instâncias EC2 executando Docker Engine, com controle total.

Construindo seu Pipeline: O Cinto de Ferramentas AWS

A AWS oferece um conjunto completo de serviços nativos para implementar pipelines CI/CD robustos, desde o controle de versão até o deploy automatizado.



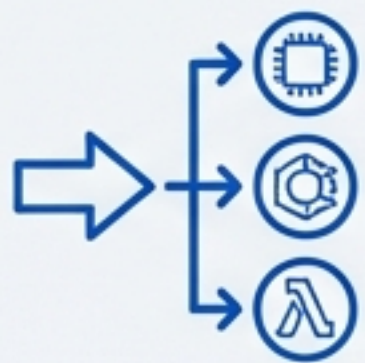
AWS CodeCommit

Repositório Git privado e totalmente gerenciado. Armazena código-fonte com alta disponibilidade e integração nativa ao IAM.



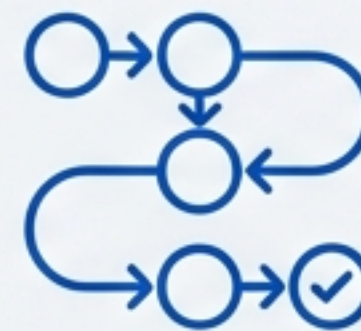
AWS CodeBuild

Serviço de build que compila código, executa testes e gera artefatos prontos para deploy. Cobra por minuto de build.



AWS CodeDeploy

Automatiza deploys de aplicações em EC2, ECS, Lambda. Suporta estratégias como Blue/Green e Rolling deployments.



AWS CodePipeline

Serviço de orquestração que conecta todos os estágios do pipeline, modelando visualmente o fluxo do commit até a produção.

Escolhendo seu Caminho: AWS Nativas vs. Ferramentas de Terceiros



Soluções AWS Nativas

Vantagens

- **Integração perfeita** com serviços AWS.
- **Sem necessidade** de gerenciar infraestrutura.
- **Segurança** e compliance nativos.
- Modelo de **pagamento** por uso.

Considerações

- **Lock-in** ao ecossistema AWS.
- Curva de aprendizado específica.
- Menor flexibilidade em workflows complexos.



Ferramentas Terceiras (Jenkins, GitLab CI, GitHub Actions)

Vantagens

- Maior **flexibilidade** e customização.
- **Comunidade** ampla e plugins diversos.
- **Portabilidade** entre clouds.
- **Experiência** consolidada no mercado.

Trade-offs

- **Necessidade** de gerenciar infraestrutura (exceto SaaS).
- Configuração de integração com AWS.
- **Custos** podem variar significativamente.

Guardando a Fortaleza: Os Seis Pilares da Segurança no ECR



Princípio do Menor Privilégio

Configure políticas IAM que concedem apenas as permissões estritamente necessárias. Separe permissões de leitura (pull) de escrita (push).



Scanning de Vulnerabilidades

Habilite o scan automático de imagens ao fazer push. O ECR usa CVE databases para identificar vulnerabilidades.



Imutabilidade de Tags

Ative a imutabilidade de tags para evitar que uma tag existente (ex: `v1.0`) seja sobrescrita, garantindo rastreabilidade.



Políticas de Lifecycle

Configure regras para remover automaticamente imagens antigas ou não-taggeadas, reduzindo custos.



Criptografia em Repouso

Utilize AWS KMS para criptografar imagens com suas próprias chaves gerenciadas, aumentando o compliance.



Auditoria com CloudTrail

Integre com AWS CloudTrail para logar todas as operações de API, criando uma trilha de auditoria completa.

Operação Inteligente: Custos e Otimização no ECR

Modelo de Precificação

\$0.10

por GB/Mês
(Custo de armazenamento)

\$0.09

por GB Transferido
(Pulls externos)

\$0.00

Dentro da Mesma Região
(Transferências internas são gratuitas)

Estratégias de Otimização

Objetivo 1: Reduzir Tamanho

- 📄 Use imagens base Alpine ou Distroless.
- 📄 Implemente Multi-stage builds.
- 📄 Remova dependências de desenvolvimento.

Objetivo 2: Gerir o Ciclo de Vida

- 🕒 Defina políticas para remover imagens antigas.
- 📅 Mantenha apenas N versões mais recentes.
- 🔄 Delete imagens não-taggeadas após X dias.

Exemplo Real

Uma empresa com 50 repositórios, média de 10 imagens por repositório (500MB cada), pagaria aproximadamente \$250/mês. Com políticas de lifecycle agressivas, esse custo pode cair para menos de \$100/mês.

A Jornada Completa, em Retrospectiva

Fundamentos CI/CD

Compreendemos a diferença entre Integração e Entrega Contínua.

Autenticação

Dominamos o processo de obter credenciais temporárias e autenticar o Docker client.

Próximos Passos

Conhecemos o ecossistema de serviços nativos e as melhores práticas.

Arquitetura do Pipeline

Mapeamos o fluxo completo: Código → Build → Teste → Registro → Deploy.

Tagging & Push

Aprendemos a estruturar URIs do ECR e a enviar imagens para a nuvem.

Próximos Passos

Conhecemos o ecossistema de serviços nativos e as melhores práticas.

Você Está Pronto.

Você completou um ciclo fundamental do DevOps moderno. Agora você compreende não apenas os conceitos teóricos de CI/CD, mas possui as habilidades práticas para implementar pipelines reais usando Docker, AWS CLI e ECR.

O fluxo DevOps está em suas mãos. Explore, experimente e construa com confiança.

